CS336 Assignment 1 (basics): Building a Transformer Language Model

Erix Liu
liuhelei.erix@gmail.com

Spring 2026

Contents

1 Byte-Pair Encoding (BPE) Tokenizer **3**
Problem: Understanding Unicode (1 point) . . . 3
Problem: Unicode Encodings (3 points) . . . 3
Problem: BPE Tokenizer Training (15 points) . . . 4
Problem: BPE Training on TinyStories (2 points) . . . 4
Problem: BPE Training on OpenWebText (2 points) . . . 4
Problem: Implementing the tokenizer (15 points) . . . 5
Problem: Experiments with tokenizers (4 points) . . . 5

2 Transformer Language Model Architecture **6**
Problem: Implementing the linear module (1 point) . . . 6
Problem: Implement the embedding module (1 point) . . . 6
Problem: Root Mean Square Layer Normalization (1 point) . . . 6
Problem: Implement the position-wise feed-forward network (2 points) . . . 6
Problem: Implement RoPE (2 points) . . . 6
Problem: Implement softmax (1 point) . . . 7
Problem: Implement scaled dot-product attention (5 points) . . . 7
Problem: Implement causal multi-head self-attention (5 points) . . . 7
Problem: Implement the Transformer block (3 points) . . . 7
Problem: Implementing the Transformer LM (3 points) . . . 8
Problem: Transformer LM resource accounting (5 points) . . . 8

3 Training a Transformer LM **9**
Problem: Implement cross-entropy (1 point) . . . 9
Problem: Tuning the learning rate (1 point) . . . 9
Problem: Implement AdamW (2 points) . . . 10
Problem: Resource accounting for training with AdamW (2 points) . . . 10
Problem: Implement cosine learning rate schedule with warmup (1 point) . . . 13
Problem: Implement gradient clipping (1 point) . . . 13

4 Training Loop **13**
Problem: Implement data loading (2 points) . . . 13
Problem: Implement model checkpointing (1 point) . . . 14
Problem: Put it together (4 points) . . . 14

5 Generating Text **15**

Problem: Decoding (3 points) . . . 15

6 Experiments **16**
Problem: Experiment logging (3 points) . . . 16
Problem: Tune the learning rate (3 points) . . . 16
Problem: Batch size variations (1 point) . . . 18
Problem: Generate text (1 point) . . . 18
Problem: Remove RMSNorm and train (1 point) . . . 19
Problem: Implement post-norm and train (1 point) . . . 20
Problem: Implement NoPE (1 point) . . . 21
Problem: SwiGLU vs. SiLU (1 point) . . . 21
Problem: Experiment on OWT (2 points) . . . 22
Problem: Leaderboard (6 points) . . . 22

Appendix A Experiment tracking and preliminary checks **23**

Appendix B Standard-budget training protocol **24**

Appendix C Low-resource protocols and diagnostic details **25**

Appendix D Complete standard-budget generations **26**
D.1 TinyStories . . . 26
D.1.1 P1: Story opening . . . 26
D.1.2 P2: Research opening . . . 27
D.1.3 P3: Internet-history opening . . . 28
D.2 OpenWebText . . . 28
D.2.1 P1: Story opening . . . 28
D.2.2 P2: Research opening . . . 29
D.2.3 P3: Internet-history opening . . . 30

Appendix E Low-resource TinyStories generations **32**

1 Byte-Pair Encoding (BPE) Tokenizer

Problem (unicode1): Understanding Unicode (1 point)

(a) What Unicode character does `chr(0)` return?

`chr(0)` returns the Null character (U+0000), whose `repr` is `'\x00'`.

(b) How does this character's string representation (`__repr__()`) differ from its printed representation?

`__repr__()` produces a visible escape sequence `'\x00'`, while `print()` renders it as an invisible character that occupies a position but displays nothing.

(c) What happens when this character occurs in text?

The null character is invisible when printed, so `"this is a test" + chr(0) + "string"` appears as `this is a teststring` even though the string length is 21, not 20.

Problem (unicode2): Unicode Encodings (3 points)

(a) What are some reasons to prefer training our tokenizer on UTF-8 encoded bytes, rather than UTF-16 or UTF-32?

For ASCII-heavy text, UTF-8 uses one byte per character, compared with two in UTF-16 and four in UTF-32 (excluding any byte-order mark), making the initial sequences shorter and avoiding the many zero bytes introduced by these encodings. Byte-level BPE starts from the same 256 byte values for all three encodings; UTF-8 also avoids byte-order ambiguity.

(b) Why is `decode_utf8_bytes_to_str_wrong` incorrect? Provide an example input byte string that yields incorrect results.

Code 1: Incorrect UTF-8 decoding function

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])
```

The function tries to decode each byte individually, but multi-byte UTF-8 characters cannot be split across single bytes. For example, encoding the Chinese word for "hello" with UTF-8 produces `b'\xe4\xbd\xa0\xe5\xa5\xbd'`, where each character is encoded as 3 bytes; decoding any single byte raises `UnicodeDecodeError` because a continuation byte is meaningless without its leading byte.

(c) Give a two-byte sequence that does not decode to any Unicode character(s).

`b'\x80\x80'` does not decode to any Unicode character. Both bytes are continuation bytes (`0x80`–`0xBF`), which must follow a leading byte; a sequence of only continuation bytes is invalid UTF-8.

Problem (`train_bpe`): BPE Tokenizer Training (15 points)

Code 2: BPE training function

```python
def train_bpe(
    input_path: str,
    vocab_size: int,
    special_tokens: list[str],
) -> tuple[dict[int, bytes], list[tuple[bytes, bytes]]]:
    freq_table = count_pretokens(input_path, "<|endoftext|>", special_tokens)
    trainer = BPETrainer(freq_table, special_tokens)
    del freq_table
    return trainer.train(vocab_size)
```

I split the corpus at document boundaries and pre-tokenize it in parallel, then initialize the vocabulary with the special tokens and all 256 byte values. For each BPE merge, I select the most frequent adjacent pair, breaking ties lexicographically, and use an inverted index to update counts only for affected pre-tokens. Training ends when the vocabulary reaches the requested size.

Problem (`train_bpe_tinystories`): BPE Training on TinyStories (2 points)

(a) Train a byte-level BPE tokenizer on TinyStories with a maximum vocabulary size of 10,000. How much time and memory did training take? What is the longest token in the vocabulary?

Training the 10K-token TinyStories tokenizer took approximately 109 seconds and reached a maximum resident set size of 0.80 GiB. The longest non-special tokens were " accomplishment," " disappointment," and " responsibility," each 15 bytes long, which is reasonable because they are frequent words merged together with their leading spaces.

(b) Profile your code. What part of the tokenizer training process takes the most time?

Pre-tokenization was the main end-to-end bottleneck, taking 82.15 of the 108.99 seconds. Within the merging stage, repeated full scans of the pair-frequency table dominated runtime.

Problem (`train_bpe_expts_owt`): BPE Training on OpenWebText (2 points)

(a) Train a byte-level BPE tokenizer on OpenWebText with a maximum vocabulary size of 32,000. What is the longest token in the vocabulary?

Training the 32K-token OpenWebText tokenizer took 23,321.57 seconds and peaked at 6.89 GiB of resident memory. The longest non-special tokens are tied at 64 bytes: a run of hyphens, likely used as a separator, and 16 repetitions of the byte pattern `C3 83 C3 82`, which appears to reflect character-encoding artifacts in web text.

(b) Compare and contrast the tokenizer that you get training on TinyStories versus OpenWebText.

The TinyStories tokenizer's longest tokens are ordinary story words with leading spaces, such as " accomplishment," whereas OpenWebText also learns web and topical fragments such as `http`, `https`, and " government," along with formatting and encoding artifacts. This difference reflects both the broader OpenWebText corpus and its larger 32K vocabulary, so it cannot be attributed to the corpus alone.

Problem (`tokenizer`): Implementing the tokenizer (15 points)

Encoding recognizes the longest matching special token, then applies BPE independently within each GPT-2 pre-token:

$$\text{text} \longrightarrow \text{pre-tokens} \longrightarrow \text{UTF-8 bytes} \longrightarrow \text{ranked merges} \longrightarrow \text{token IDs}.$$

The merge loop always selects the adjacent pair with the smallest training rank, stopping when no applicable merge remains.

Code 3: BPE encoding within one pre-token

```
while len(pretoken) >= 2:
    selected_pair = self._find_best_merge_pair(pretoken)
    if selected_pair is None:
        break
    pretoken = self._merge_selected_pair(pretoken, selected_pair)
```

The streaming encoder retains unfinished pre-tokens and possible special-token prefixes across chunks. Decoding joins bytes before interpreting UTF-8, because one character can span several tokens:

Code 4: Byte-level decoding

```
token_bytes = b"".join(self.vocab[token_id] for token_id in ids)
return token_bytes.decode("utf-8", errors="replace")
```

Problem (`tokenizer_experiments`): Experiments with tokenizers (4 points)

(a) Sample 10 documents from TinyStories and OpenWebText. What is each tokenizer's compression ratio (bytes/token)?

Excluding document separators, the TinyStories tokenizer achieved 4.019 bytes per token on 10 sampled TinyStories documents (8,484 bytes / 2,111 tokens). The OpenWebText tokenizer achieved 3.774 bytes per token on 10 sampled OpenWebText documents (93,516 bytes / 24,778 tokens).

(b) What happens if you tokenize your OpenWebText sample with the TinyStories tokenizer?

On the same 10 OpenWebText documents, the TinyStories tokenizer achieved 2.816 bytes/token (93,516 bytes / 33,213 tokens), compared with 3.774 bytes/token for the OpenWebText tokenizer. It used about 34% more tokens for this sample.

(c) Estimate the throughput of your tokenizer (bytes/second). How long would it take to tokenize the Pile (825GB)?

On my machine (macOS + Apple M4 + 16GB RAM), the OpenWebText tokenizer encoded the cached 10-document sample at about 1.67 MB/s (1,668,392 bytes/s) over repeated in-memory passes. At that rate, tokenizing 825 GB would take about 137 hours (5.7 days), excluding file I/O and variation across documents.

(d) Using the tokenizers to encode the respective training and development datasets into a sequence of integer token IDs. Why serializing the token IDs as a NumPy array of datatype `uint16` is an appropriate choice?

The TinyStories and OpenWebText vocabularies use IDs 0–9,999 and 0–31,999, respectively, both within the `uint16` range of 0–65,535. Each ID therefore occupies 2 bytes rather than 4 bytes with `uint32`, reducing storage and memory-mapped I/O for the tokenized datasets.

2 Transformer Language Model Architecture

Problem (`linear`): Implementing the linear module (1 point)

The bias-free weight has shape $(d_{\text{out}}, d_{\text{in}})$; an Einstein summation projects only the final input dimension.

Code 5: Linear projection

```
def forward(self, x: Tensor) -> Tensor:
    return einsum(x, self.weight, "... d_in, d_out d_in -> ... d_out")
```

Problem (`embedding`): Implement the embedding module (1 point)

A trainable (V, d_{model}) table is initialized from $\mathcal{N}(0,1)$ truncated to $[-3,3]$. Lookup adds an embedding dimension to the input shape:

$$E \in \mathbb{R}^{V \times d_{\text{model}}}, \qquad Y_{\ldots,:} = E_{\text{token_ids}_{\ldots},:}.$$

Code 6: Embedding lookup

```
def forward(self, token_ids: Tensor) -> Tensor:
    return self.weight[token_ids]
```

Problem (`rmsnorm`): Root Mean Square Layer Normalization (1 point)

I compute $\text{RMSNorm}(x) = g \odot x / \sqrt{\text{mean}(x^2) + \epsilon}$ over the final dimension, with g initialized to ones. The reduction uses float32 for numerical stability, and the result is cast back to the input dtype.

Code 7: RMSNorm forward pass

```
in_dtype = x.dtype
x = x.to(torch.float32)
rms = torch.sqrt(reduce(x**2, "... d_model -> ... 1", "mean") + self.eps)
return (x / rms * self.weight).to(in_dtype)
```

Problem (`positionwise_feedforward`): Implement the position-wise feed-forward network (2 points)

Three bias-free projections implement $W_2(\text{SiLU}(W_1 x) \odot W_3 x)$ independently at each token, where $\text{SiLU}(x) = x\sigma(x)$. My default hidden width rounds $8d_{\text{model}}/3$ to the nearest multiple of 64; an explicit `d_ff` overrides it.

Code 8: SwiGLU computation

```
def swiglu(x: Tensor, w1: Linear, w2: Linear, w3: Linear) -> Tensor:
    return w2(silu(w1(x)) * w3(x))
```

Problem (`rope`): Implement RoPE (2 points)

For position p and adjacent pair index $i = 0, \ldots, d_k/2 - 1$, RoPE applies the rotation

$$\phi_{p,i} = p\theta^{-2i/d_k}, \qquad \begin{pmatrix} a' \\ b' \end{pmatrix} = \begin{pmatrix} \cos\phi_{p,i} & -\sin\phi_{p,i} \\ \sin\phi_{p,i} & \cos\phi_{p,i} \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix}.$$

Sine and cosine tables are cached as nonpersistent buffers and indexed by the supplied token positions. Rotation uses float32 arithmetic before casting back to the input dtype.

Problem (`softmax`): Implement softmax (1 point)

I move the requested dimension to the last axis, subtract its maximum, exponentiate, and divide by the sum before restoring the original axis order. Subtracting the maximum avoids exponential overflow; a slice containing only $-\infty$ still produces NaNs.

Code 9: Softmax computation

```python
def softmax(x: Tensor, dim: int) -> Tensor:
    moved = x.movedim(dim, -1)
    shifted = moved - reduce(moved, "... feature -> ... 1", "max")
    exp_x = torch.exp(shifted)
    result = exp_x / reduce(exp_x, "... feature -> ... 1", "sum")
    return result.movedim(-1, dim)
```

Problem (`scaled_dot_product_attention`): Implement scaled dot-product attention (5 points)

Attention normalizes scaled dot products over keys, then forms a weighted sum of values:

$$S = \frac{QK^\top}{\sqrt{d_k}}, \qquad \widetilde{S}_{ij} = \begin{cases} S_{ij}, & M_{ij} = \text{True}, \\ -\infty, & M_{ij} = \text{False}, \end{cases} \qquad Y = \text{softmax}(\widetilde{S})V.$$

Einstein summations preserve arbitrary leading batch dimensions. The mask broadcasts to the score shape; each query must retain at least one allowed finite score.

Problem (`multihead_self_attention`): Implement causal multi-head self-attention (5 points)

Separate projections produce Q, K, V; each is split into h heads of width $d_k = d_{\text{model}}/h$:

$$(\ldots, T, d_{\text{model}}) \longrightarrow (\ldots, h, T, d_k).$$

RoPE rotates Q and K, using positions $0, \ldots, T-1$ when none are provided. A lower-triangular mask enforces causality:

$$M_{ij} = [j \le i], \qquad H_r = \text{Attention}(Q_r, K_r, V_r; M), \qquad Y = \text{Concat}(H_1, \ldots, H_h)W_O^\top.$$

Problem (`transformer_block`): Implement the Transformer block (3 points)

Each block uses pre-normalization and two residual connections:

$$y = x + \text{Attention}(\text{RMSNorm}_1(x)),$$
$$z = y + \text{FFN}(\text{RMSNorm}_2(y)).$$

Both sublayers preserve the input shape.

Code 10: Pre-norm Transformer block

```python
def forward(self, x: Tensor) -> Tensor:
    y = x + self.attn(self.ln1(x))
    return y + self.ffn(self.ln2(y))
```

Problem (`transformer_lm`): Implementing the Transformer LM (3 points)

The forward pass maps token IDs to unnormalized vocabulary logits:

$$\begin{aligned} H_0 &= \text{Embedding}(X), \\ H_\ell &= \text{Block}_\ell(H_{\ell-1}), \quad \ell = 1, \ldots, L, \\ Z &= \text{RMSNorm}_{\text{final}}(H_L) W_{\text{out}}^\top \in \mathbb{R}^{B \times T \times V}. \end{aligned}$$

Blocks have independent trainable parameters and share one fixed RoPE cache. Input embeddings and the output projection use separate weights; softmax is deferred to the loss or sampling routine.

Problem (`transformer_accounting`): Transformer LM resource accounting (5 points)

Let $V = 50{,}257$, $T = 1{,}024$, and let L, d, h, f denote the number of layers, model width, number of heads, and FFN width, respectively. For XL, $(L, d, h, f) = (48, 1600, 25, 4288)$; FLOPs below count dense forward matrix multiplies for one sequence, including masked entries, using $2mnp$ FLOPs for an $(m \times n)(n \times p)$ product.

(a) How many trainable parameters would a GPT-2 XL-sized model have? How much memory is required to load it (single-precision)?

With untied embedding/output weights, no biases, and two RMSNorms per block, the parameter count is

$$P = 2Vd + L(4d^2 + 3df + 2d) + d = 1{,}640{,}452{,}800.$$

FP32 parameters require $4P = 6{,}561{,}811{,}200$ bytes, approximately 6.562 GB (6.111 GiB), excluding buffers and activations.

(b) Identify the matrix multiplies required for a forward pass. How many FLOPs do they require in total?

The matrix multiplies and their costs are listed below; per-layer attention costs are summed over all heads, where $d_k = d/h$.

Component	Matrix products	FLOPs per layer
Q, K, V projections	$3 \times (T, d)(d, d)$	$6Td^2$
Attention scores	$h \times (T, d_k)(d_k, T)$	$2T^2d$
Attention-weighted values	$h \times (T, T)(T, d_k)$	$2T^2d$
Output projection	$(T, d)(d, d)$	$2Td^2$
SwiGLU	$2 \times (T, d)(d, f) + (T, f)(f, d)$	$6Tdf$
LM head (once per model)	$(T, d)(d, V)$	$2TdV$

Thus the total is

$$F = L(8Td^2 + 4T^2d + 6Tdf) + 2TdV = 3{,}516{,}769{,}894{,}400 \approx 3.517 \text{ TFLOPs}.$$

Embedding lookup and elementwise operations are excluded.

(c) Which parts of the model require the most FLOPs?

SwiGLU dominates at 57.53% of total FLOPs, followed by QKV and output projections together at 28.62%. The two attention matrix multiplies account for 9.16%, and the LM head accounts for 4.68%.

(d) Repeat the analysis with GPT-2 small, medium, and large. How do proportional FLOPs change with model size?

Using f equal to $8d/3$ rounded to the nearest multiple of 64, the component shares are:

Model (f)	TFLOPs	QKV	Output	Attention	FFN	LM head
Small (2048)	0.292	14.91%	4.97%	13.25%	39.76%	27.10%
Medium (2752)	0.830	18.62%	6.21%	12.42%	50.05%	12.70%
Large (3392)	1.769	20.49%	6.83%	10.93%	54.30%	7.45%
XL (4288)	3.517	21.47%	7.16%	9.16%	57.53%	4.68%

Here "Attention" combines $QK^\top$ and the attention-weighted values. At fixed T, V and $f \approx 8d/3$, projections and FFN scale as Ld^2, attention as Ld, and the LM head as d; increasing model depth and width therefore raises the projection/FFN shares and lowers the attention/LM-head shares.

(e) Take GPT-2 XL and increase the context length to 16,384. How does the total FLOPs and relative contribution change?

Increasing T by 16× multiplies projection, FFN, and LM-head FLOPs by 16, but attention FLOPs by 256:

$$F_{\text{new}} = 16(F_{\text{old}} - F_{\text{attn}}) + 256F_{\text{attn}} \approx 133.578 \text{ TFLOPs} = 37.983\, F_{\text{old}},$$

where $F_{\text{attn}} = 4LT^2d$ at the original context length. Attention becomes dominant at 61.73% (up from 9.16%), while QKV, output projection, FFN, and LM-head shares fall to 9.04%, 3.01%, 24.24%, and 1.97%, respectively.

3 Training a Transformer LM

Problem (`cross_entropy`): Implement cross-entropy (1 point)

Subtracting each prediction's maximum logit avoids exponential overflow without changing the loss. For n predictions and targets y_b,

$$s_{bj} = z_{bj} - \max_k z_{bk}, \qquad \mathcal{L} = \frac{1}{n}\sum_{b=1}^{n}\left[\log\sum_j \exp(s_{bj}) - s_{b,y_b}\right].$$

The target lookup retains a singleton vocabulary axis for matching shapes:

Code 11: Stable cross-entropy after shifting logits

```
logits_sum = reduce(torch.exp(shifted), "... vocab_size -> ... 1", "sum")
target_logits = shifted.gather(-1, targets.unsqueeze(-1))
return (torch.log(logits_sum) - target_logits).mean()
```

The provided test passed, including its large-logit case.

Problem (`learning_rate_tuning`): Tuning the learning rate (1 point)

Compared with the slow loss decrease at $\alpha = 1$, $\alpha = 10$ decreased the loss faster and $\alpha = 100$ reached approximately zero within 10 iterations (the first update left the loss essentially unchanged). At $\alpha = 1000$, the loss diverged, increasing from about 24.17 to 6.35×10^{19} after 10 updates.

Initial learning rate	Initial loss	Loss after 10 updates
1	24.1693	19.7481
10	24.1693	2.8504
100	24.1693	3.8487×10^{-24}
1000	24.1693	6.3511×10^{19}

A local random generator with seed 42 sampled a single 10×10 initial weight matrix. Each run cloned this matrix and created a fresh SGD optimizer, holding the initialization, objective, precision, and number of updates fixed while varying only the initial learning rate. All runs used the decaying step size $\alpha/\sqrt{t+1}$ from the SGD example; losses were recorded initially and after each of 10 updates.

Problem (`adamw`): Implement AdamW (2 points)

Subclass `torch.optim.Optimizer`, validate the hyperparameters, and store their defaults in parameter groups. For each parameter with a gradient, lazily initialize its first and second moment tensors to zero and maintain a separate update counter starting at 1. At update t, compute

$$\alpha_t = \alpha \frac{\sqrt{1-\beta_2^t}}{1-\beta_1^t}, \qquad m \leftarrow \beta_1 m + (1-\beta_1) g, \qquad v \leftarrow \beta_2 v + (1-\beta_2) g^2.$$

Apply decoupled weight decay before the moment-adjusted parameter update:

$$\theta \leftarrow (1-\alpha\lambda)\theta, \qquad \theta \leftarrow \theta - \alpha_t \frac{m}{\sqrt{v}+\epsilon}.$$

The update runs under `torch.no_grad()`; the decay factor is computed once per parameter group using its current learning rate:

Code 12: In-place AdamW update for one parameter

```
lr_t = lr * sqrt(1 - beta_2**t) / (1 - beta_1**t)
p.mul_(decay_factor)
m.mul_(beta_1).add_(grad, alpha=1 - beta_1)
v.mul_(beta_2).addcmul_(grad, grad, value=1 - beta_2)
denom = v.sqrt().add_(eps)
p.addcdiv_(m, denom, value=-lr_t)
```

Taking a separate square-root tensor preserves the stored second moment. An optional closure is evaluated with gradients enabled, and its loss is returned.

Problem (`adamw_accounting`): Resource accounting for training with AdamW (2 points)

(a) How much peak memory does running AdamW require? Decompose into parameters, activations, gradients, and optimizer state.

Let B denote batch size, T context length, V vocabulary size, L the number of layers, d model width, h the number of attention heads, and $f = \frac{8}{3}d$ the feed-forward width. Assume untied input and output embeddings, bias-free projections, and 4 bytes per FP32 element.

The input embedding and LM head contribute $2Vd$ parameters. Each block has $4d^2$ attention weights, $3df$ feed-forward weights, and $2d$ RMSNorm weights; the final RMSNorm adds another d. Thus the parameter count is

$$P = 2Vd + L(4d^2 + 3df + 2d) + d = 2Vd + L(12d^2 + 2d) + d.$$

Parameters and gradients each occupy $4P$ bytes. AdamW's two moment tensors occupy $8P$ bytes; per-parameter counters are neglected.

For activation accounting, retain one output for each listed operator, without checkpointing or fused attention. The per-block element counts are:

Operator outputs	Elements per block
Two RMSNorms	$2BTd$
Q, K, V projections	$3BTd$
Attention scores	BhT^2
Softmax probabilities	BhT^2
Weighted sum of values	BTd
Attention output projection	BTd
FFN gate and value projections	$2BTf$
SiLU on the gate branch	BTf
Elementwise gate product	BTf
FFN output projection	BTd

Each head produces its own $T \times T$ score matrix, giving BhT^2 rather than BT^2 elements. Summing the table gives $BT(8d + 4f) + 2BhT^2$ elements per block. Outside the blocks, count BTd elements for final RMSNorm, BTV logits, and one BTV log-probability tensor for cross-entropy. The total activation element count is therefore

$$\begin{aligned} A(B) &= B\left[L\left(T(8d + 4f) + 2hT^2\right) + Td + 2TV\right] \\ &= B\left[L\left(\frac{56}{3}Td + 2hT^2\right) + Td + 2TV\right]. \end{aligned}$$

The resulting memory estimate, in bytes, is

$$\boxed{M(B) = \underbrace{4P}_{\text{parameters}} + \underbrace{4P}_{\text{gradients}} + \underbrace{8P}_{\text{AdamW state}} + \underbrace{4A(B)}_{\text{activations}}.}$$

This is a simplified retained-output estimate rather than an allocator peak measurement. It excludes residual additions, RoPE, masks, RMSNorm statistics, and implementation-specific temporary buffers; actual backward storage and buffer reuse can change the peak.

(b) Instantiate your answer for GPT-2 XL. What is the maximum batch size within 80GB?

Substitute $V = 50{,}257$, $T = 1{,}024$, $L = 48$, $d = 1{,}600$, and $h = 25$. Use the exact algebraic ratio $f = \frac{8}{3}d$ requested in this problem, rather than the rounded width of 4,288 in the earlier accounting problem. This gives $P = 1{,}635{,}537{,}600$ and the following memory breakdown, using $1\,\text{GB} = 10^9$ bytes:

Component	Memory (GB)
Parameters	6.542150400
Gradients	6.542150400
AdamW moments	13.084300800
Activations	$16.356614144B$

Consequently,

$$\boxed{M(B) = 16.356614144B + 26.168601600 \quad \text{GB},} \qquad B_{\max} = \left\lfloor \frac{80 - 26.168601600}{16.356614144} \right\rfloor = \boxed{3}.$$

The estimate is 75.238444032 GB for $B = 3$ and 91.595058176 GB for $B = 4$. If the rounded width $f = 4{,}288$ is used instead, then $P = 1{,}640{,}452{,}800$ and $M(B) = 16.373391360B + 26.247244800$ GB; the maximum batch size remains 3.

(c) How many FLOPs does running one step of AdamW take?

Count scalar additions, subtractions, multiplications, divisions, and square roots as one operation each. Exclude forward/backward computation and the scalar hyperparameter arithmetic that does not scale with the number of parameter elements. A literal expansion of the algorithm has the following costs per parameter element:

Operation	FLOPs per element
Weight decay: $\theta - (\alpha\lambda)\theta$	2
First moment: $\beta_1 m + (1-\beta_1)g$	3
Second moment: $\beta_2 v + (1-\beta_2)g^2$	4
Parameter update: $\theta - \alpha_t m/(\sqrt{v}+\epsilon)$	5

Thus,

$$F_{\text{AdamW}} \approx 14P = \boxed{22{,}897{,}526{,}400 \text{ FLOPs}}.$$

The implementation precomputes $1-\alpha\lambda$ once per parameter group, so its decay operation needs only one multiplication per element. Under the same counting convention, this reduces its cost to $13P = 21{,}261{,}988{,}800$ FLOPs. The time estimate below uses the literal $14P$ convention consistently.

(d) Assuming 50% MFU on an H100, how long would it take to train GPT-2 XL for 400K steps with batch size 1024?

Count the dominant dense matrix multiplications in a forward pass, with two FLOPs per multiply-accumulate. Per sequence, the attention projections cost $8Td^2$ per layer, the two attention matrix multiplications cost $4T^2d$, and the three FFN projections cost $6Tdf$. The LM head adds $2TdV$. Therefore,

$$\begin{aligned} F_{\text{forward},1} &= L(8Td^2 + 4T^2d + 6Tdf) + 2TdV \\ &= L(24Td^2 + 4T^2d) + 2TdV \\ &= 3{,}506{,}703{,}564{,}800 \text{ FLOPs}. \end{aligned}$$

This approximation omits elementwise forward operations. For $B = 1{,}024$, assuming backward costs twice forward, one training step costs

$$F_{\text{step}} = 3BF_{\text{forward},1} + 14P = 1.0772616248592 \times 10^{16} \text{ FLOPs}.$$

The effective throughput at 50% MFU is $0.5 \times 495 \times 10^{12} = 2.475 \times 10^{14}$ FLOP/s. Hence,

$$t_{\text{step}} \approx 43.525722217 \text{ s}, \qquad t_{\text{total}} = \frac{400{,}000F_{\text{step}}}{(2.475 \times 10^{14})\,3600} \approx \boxed{4{,}836.19 \text{ hours}} \approx 201.51 \text{ days}.$$

The stipulated batch size of 1,024 exceeds the single-device memory estimate in part (b); this is a throughput-based estimate assuming sufficient memory or gradient accumulation to achieve the effective batch size. Using the rounded FFN width of 4,288 instead gives approximately 4,850.07 hours.

Problem (`learning_rate_schedule`): Implement cosine learning rate schedule with warmup (1 point)

Return a scalar learning rate using three branches: linear warmup for $t < T_w$, cosine decay for $T_w \le t \le T_c$, and the minimum learning rate for $t > T_c$:

$$\alpha(t) = \begin{cases} \alpha_{\max}\, t/T_w, & t < T_w, \\ \alpha_{\min} + \frac{1}{2}(\alpha_{\max} - \alpha_{\min}) \left[1 + \cos\left(\pi \frac{t - T_w}{T_c - T_w}\right)\right], & T_w \le t \le T_c, \\ \alpha_{\min}, & t > T_c. \end{cases}$$

Here T_c is the iteration at which decay ends, so the cosine interval has length $T_c - T_w$. Its endpoints are $\alpha(T_w) = \alpha_{\max}$ and $\alpha(T_c) = \alpha_{\min}$, assuming $T_c > T_w$.

Problem (`gradient_clipping`): Implement gradient clipping (1 point)

Collect references to all nonempty parameter gradients, allowing repeated traversal even when the parameters are supplied as an iterator. Return immediately if there are no gradients. Compute a single global norm over all parameter elements,

$$G = \sqrt{\sum_{p:\, p.\mathrm{grad} \neq \mathrm{None}} \sum_i (p.\mathrm{grad}_i)^2}.$$

With maximum norm M and $\epsilon = 10^{-6}$, apply one shared scale:

$$g_p \leftarrow \begin{cases} \frac{M}{G + \epsilon} g_p, & G \ge M, \\ g_p, & G < M. \end{cases}$$

Scaling all gradients in place bounds the combined norm while preserving the direction of the concatenated gradient vector.

4 Training Loop

Problem (`data_loading`): Implement data loading (2 points)

Sample B starting indices independently and uniformly with replacement. For N dataset tokens and context length T,

$$s_b \sim \mathrm{Uniform}\{0, \ldots, N - T - 1\}, \qquad X_{b,t} = x_{s_b + t}, \quad Y_{b,t} = x_{s_b + t + 1}, \quad 0 \le t < T.$$

Thus X and Y have shape (B, T), with targets shifted one token forward.

Code 13: Sampling input and target windows

```
starts = np.random.randint(0, len(dataset) - context_length, size=batch_size)
inputs_np = np.stack([dataset[s:s + context_length] for s in starts])
targets_np = np.stack([dataset[s + 1:s + context_length + 1] for s in starts])
inputs = torch.tensor(inputs_np, dtype=torch.long, device=device)
targets = torch.tensor(targets_np, dtype=torch.long, device=device)
```

Only sampled windows are materialized; the dataset can remain memory-mapped.

Problem (`checkpointing`): Implement model checkpointing (1 point)

A `torch.save` dictionary contains the following state:

Field	Purpose
`model`, `optimizer`	Restore weights and AdamW state
`iteration`	Resume at the next unfinished update
`model_config` (optional)	Reconstruct the model for generation
`numpy_rng_state`, `torch_rng_state`	Restore CPU random generators
`mps_rng_state`, `cuda_rng_states`	Restore available accelerator generators

Loading restores model, optimizer, and random generator states and returns the iteration count. The training script always supplies the model configuration, enabling inference to reconstruct the architecture:

Code 14: Reconstructing a model from a training checkpoint

```
checkpoint = torch.load(path, map_location="cpu", weights_only=False)
model = TransformerLM(**checkpoint["model_config"])
model.load_state_dict(checkpoint["model"])
```

Problem (`training_together`): Put it together (4 points)

The training script exposes data paths, model and optimizer hyperparameters, device, random seed, logging intervals, and checkpoint paths through command-line arguments. Training and validation token files are opened as read-only `uint16` NumPy memory maps to avoid loading the entire corpora into memory.

Each update sets the warmup/cosine learning rate before computing the loss. The core update, after setting the learning rate, is:

Code 15: Training update

```
optimizer.zero_grad()
inputs, targets = get_batch(
    train_dataset, args.batch_size, args.context_length, args.device
)
logits = model(inputs)
loss = cross_entropy(
    logits.reshape(-1, logits.shape[-1]), targets.reshape(-1)
)
loss.backward()
gradient_clipping(model.parameters(), args.max_l2_norm)
optimizer.step()
```

Validation averages losses over sampled batches under `eval()` with gradients disabled, then restores the previous mode. Configuration, training loss, validation loss, and learning rate are logged to the console and `train.log`. Checkpoints are saved periodically and at the end; resuming restores saved states using the supplied training configuration.

5 Generating Text

Problem (`decoding`): Decoding (3 points)

Generation handles a single non-empty prompt. With context length C, each step uses only the most recent window:

$$v = \mathrm{TransformerLM}\big(x_{\max(1,t-C+1):t}\big)_{\mathrm{last},:}, \qquad q_i = \frac{\exp(v_i/\tau)}{\sum_j \exp(v_j/\tau)}, \quad \tau > 0.$$

Nucleus sampling. Sort q in descending order and let $c_i = \sum_{j \le i} q_{(j)}$. Keep the smallest prefix reaching probability p, including the token that first reaches or crosses the threshold:

$$m_i = [c_i - q_{(i)} < p], \qquad \widetilde{q}_{(i)} = \frac{m_i q_{(i)}}{\sum_j m_j q_{(j)}}.$$

Code 16: Top-p filtering and token-ID recovery

```
sorted_probs, sorted_ids = torch.sort(probs, dim=-1, descending=True)
cumulative_probs = sorted_probs.cumsum(dim=-1)
keep_mask = (cumulative_probs - sorted_probs) < top_p
filtered_probs = sorted_probs.masked_fill(~keep_mask, 0.0)
filtered_probs /= filtered_probs.sum(dim=-1, keepdim=True)
sampled_index = torch.multinomial(filtered_probs, num_samples=1)
next_token_id = sorted_ids[sampled_index]
```

Here `probs` is the temperature-scaled softmax distribution. The helper also supports greedy selection at $\tau = 0$; the CLI requires $\tau > 0$.

Autoregressive loop. Under `torch.inference_mode()`, the loop appends each sampled token and stops at EOS or the length limit:

Code 17: Generation loop for one prompt

```
for _ in range(max_new_tokens):
    input_ids = torch.tensor(
        history[-context_length:], dtype=torch.long, device=device
    ).unsqueeze(0)
    logits = model(input_ids)[0, -1, :]
    token_id = sample_next_token(logits, temperature, top_p).item()
    history.append(token_id)
    generated_ids.append(token_id)
    if token_id == eos_token_id:
        break
```

The function returns new IDs only, including any sampled EOS. The entry point reconstructs the model from its checkpoint, sets evaluation mode and the random seed, and encodes the prompt. Before displaying the result:

Code 18: Removing EOS before decoding the completion

```
if generated_ids and generated_ids[-1] == args.eos_token_id:
    generated_ids = generated_ids[:-1]
print(args.prompt + tokenizer.decode(generated_ids))
```

6 Experiments

Problem (`experiment_log`): Experiment logging (3 points)

Each run records configuration, losses, LR, steps/tokens, elapsed time, and checkpoints in `train.log` and W&B. The low-resource TinyStories experiments use MPS and about 40.96M tokens; the added TinyStories/OWT main runs use CUDA and 327.68M tokens. Logging details, shared settings, and additional diagnostics are in Appendices A–C.

Problem (`learning_rate`): Tune the learning rate (3 points)

(a) Low-resource search. I screened three LRs for 1,000 steps, then compared peaks 5×10^{-4}, 10^{-3}, and 2×10^{-3} in fresh 10,000-step runs (40.96M tokens each). All use the same four-layer TinyStories model, batch 16, seed 42, and cosine floor equal to one tenth of the peak. The lowest final loss, 1.5966, came from 10^{-3} and met the 2.00 low-resource target. I retained this LR for subsequent baselines.

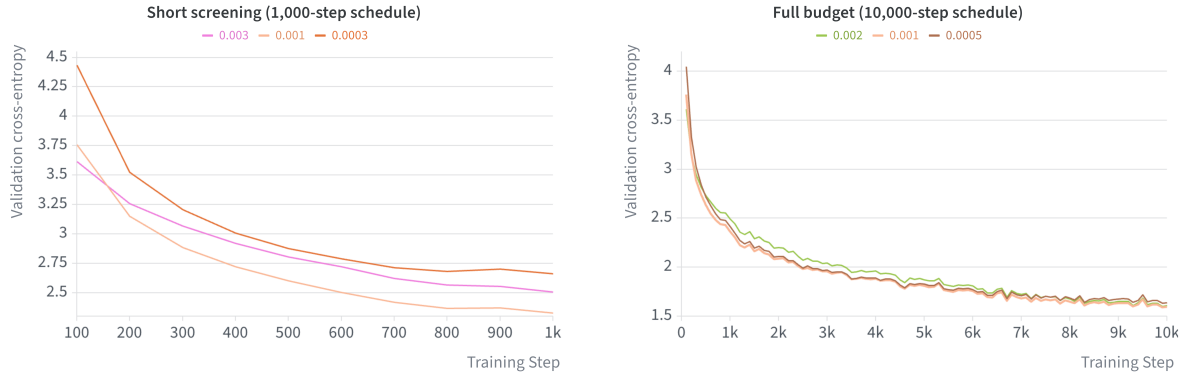


Figure 1: W&B validation curves for short screening (left) and 40.96M-token comparisons (right), with smoothing disabled. Legends show peak learning rates. The low-resource validation target is 2.00.

Peak LR	Minimum LR	Final val loss	Best sampled val loss	Final PPL
5×10^{-4}	5×10^{-5}	1.6402	1.6343	5.16
1×10^{-3}	1×10^{-4}	**1.5966**	**1.5921**	**4.94**
2×10^{-3}	2×10^{-4}	1.6092	1.6010	5.00

Table 1: Equal-budget results after 10,000 steps (40.96M tokens). Final perplexity is exp(final validation loss).

(b) Stability. Larger tested LRs first slowed learning, then caused repeated loss explosions at 0.3/0.5. Losses stayed finite and sometimes recovered; this is instability, not proven unbounded divergence. These short probes used the 10,000-step schedule and are compared over their first 1,000 updates. The best tested LR was well below the unstable range, so the search does not establish a sharp "edge of stability."

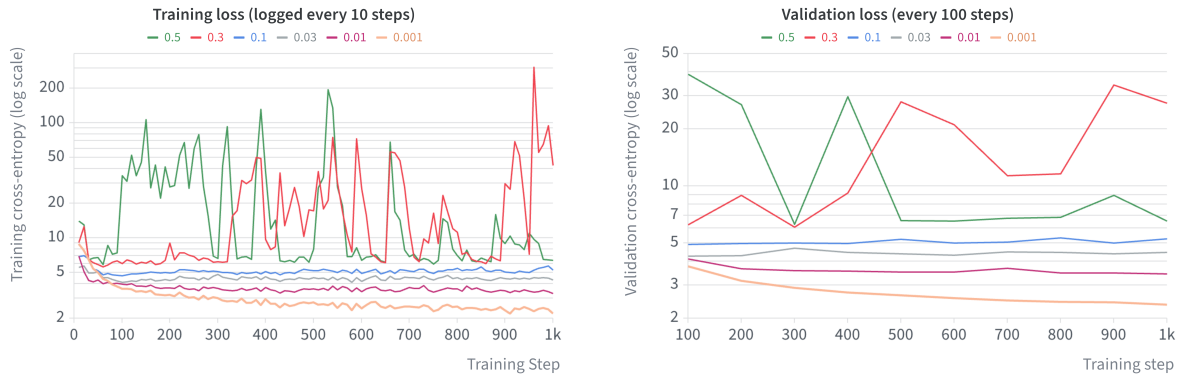


Figure 2: W&B training (left) and validation (right) curves over the first 1,000 steps, with smoothing disabled and logarithmic loss axes. Legends show peak learning rates; the 0.001 curve is the full baseline restricted to this window. Warmup lasts 100 steps.

Peak LR	Val loss at step 1,000	Observed behavior
10^{-3}	2.3642	Consistent improvement
10^{-2}	3.4377	Slow improvement
3×10^{-2}	4.4470	High-loss plateau and oscillation
10^{-1}	5.2528	High-loss plateau and oscillation
0.3	27.3223	Repeated loss explosions
0.5	6.5385	Repeated loss explosions and recovery

Table 2: High-LR probes compared at the same training step and token budget.

(c) Standard-budget TinyStories follow-up. A fresh baseline retained LR $10^{-3}/10^{-4}$ and trained for 80,000 updates (327.68M tokens), with 800-step warmup and the same fixed validation windows as the matched low-resource baseline. Final loss was 1.3245, meeting the standard target of 1.45; the sampled minimum was 1.3240 at 79,200 updates. Complete settings are in Appendix B.

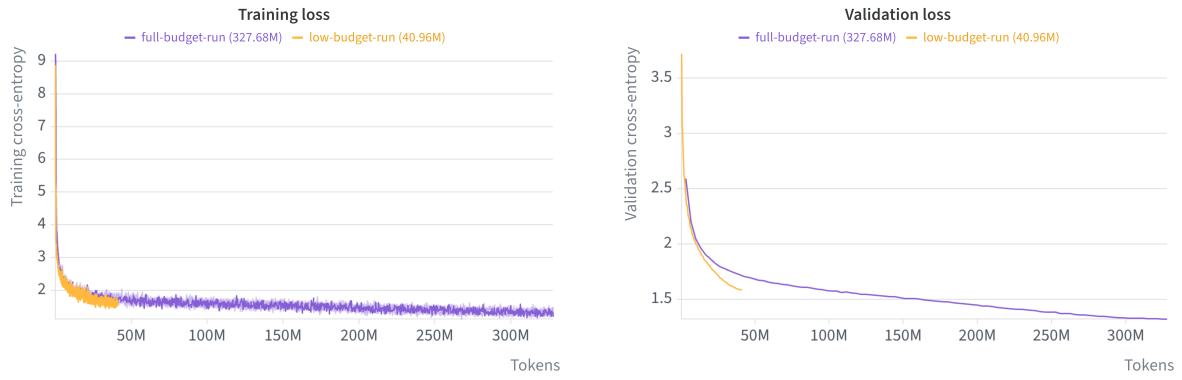


Figure 3: TinyStories training loss (left) and fixed-window validation loss (right) against training tokens. Purple: 327.68M; orange: 40.96M. W&B exports use linear axes; the short run ends at its actual budget.

The matched short run ended at 1.5879, a gap of 0.2634 nats per token. Its earlier advantage at equal token counts reverses as the long run continues. Both token budget and LR schedule changed, and TinyStories moved from MPS to CUDA; this compares complete recipes, not tokens alone. The low-resource ablations were not repeated at the larger budget.

Problem (`batch_size_experiment`): Batch size variations (1 point)

I varied batch size while keeping approximately 40.96M training tokens and LR $10^{-3}/10^{-4}$ fixed. Updates, warmup, and evaluation/saving intervals scaled inversely with batch size; all runs used the same private validation windows (seed 42, 20 batches of 16 sequences).

Batch	Updates	Warmup	Eval interval	Final val loss	Time (min)
1	160,000	1,600	1,600	1.7321	144.34
8	20,000	200	200	**1.5839**	89.51
16	10,000	100	100	1.5879	87.03
32	5,000	50	50	1.6081	85.51
48	3,333	33	33	1.6329	84.62

Table 3: Approximately equal-token, fixed-LR batch-size comparison on MPS. Warmup and evaluation intervals are in updates. Times use the last logged training elapsed time and include intervening evaluation/checkpoint work, but exclude startup and final evaluation/checkpoint overhead.

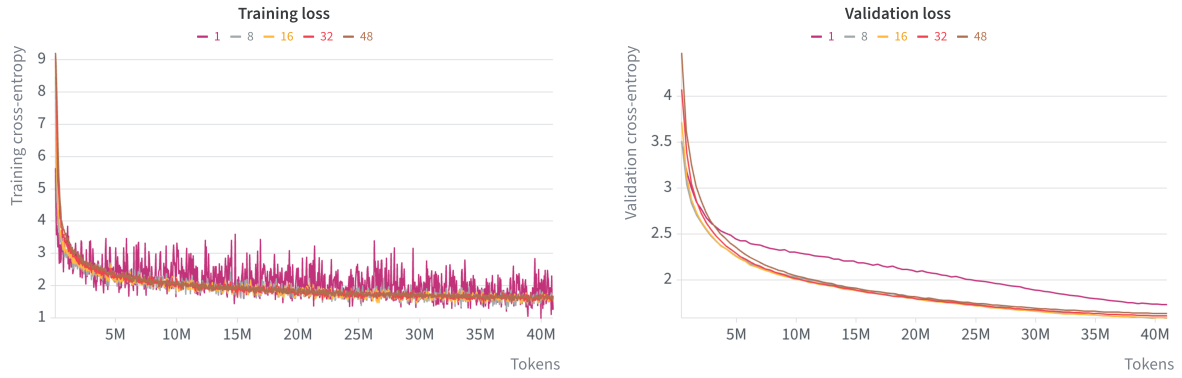


Figure 4: W&B training loss (left) and validation loss (right) against tokens processed, with smoothing disabled. Legend numbers denote batch sizes; colors match across panels: magenta 1, gray 8, orange 16, red 32, and brown 48. Training points are individual minibatch losses.

Batches 8–16 gave the best observed loss/runtime balance. Batch 8's 1.5839 was only 0.0040 below batch 16, insufficient for a reliable ranking from one seed. Batch 1 fluctuated more and learned less efficiently; batches 32/48 were slightly faster but had worse final losses. Increasing 16 to 48 saved only about 2.8% elapsed time.

These are fixed-LR recipes, not per-batch optima. Batch 1's system-wide swap growth confounds timing. Capacity probes passed at 48 but stopped at 56/64 for memory pressure, so 48 is the largest tested passing batch, not a proven limit. Diagnostics and run-selection details are in Appendix C.

Problem (`generate`): Generate text (1 point)

Low-resource samples. The matched TinyStories baseline (loss 1.5879) used the Lily prompt, seed 42, top-$p = 0.9$, and temperatures 0.7/1.0/1.3, with a maximum of 256 new tokens. All stopped at EOS. Full outputs are in Appendix E.

Lower temperatures produced readable story structure but inconsistent logic: the mother returns the ball twice at 0.7, while 1.0 shifts from a rose to an apple and a flower. At 1.3, phrases such as "grandma mommy" and "cupboardving" weaken fluency.

Temperature	Prompt tokens	Sampled tokens	Text tokens	Stop reason
0.7	13	142	141	EOS
1.0	13	207	206	EOS
1.3	13	157	156	EOS

Table 4: Counts reported by the generation CLI. Sampled tokens include the terminal EOS; text tokens exclude it. Both counts exclude the prompt. All three samples stopped at EOS before the 256-new-token limit.

Standard-budget samples. Both final models use the same three prompts:

P1: `Once upon a time, there was a little girl named Lily.`
P2: `In a recent study, researchers found that`
P3: `The history of the Internet began with`

Each prompt is sampled at 0.7/1.0/1.3 with seed 42 reset, top-$p = 0.9$, CUDA, and a 256-new-token/EOS limit. Appendix D contains all 18 outputs and token counts. The excerpts below use 0.7.

P1: story.

> **TinyStories:** She had a big box of crayons. [...] "What a nice picture!" she said.
>
> **OWT:** When she got the chance to pick up the rope, she became aware of what was going on, and was seen in a scene in a police car.

TinyStories sustains a story but confuses drawing with cake-making. OWT switches to repetitive incident reporting. P1 favors TinyStories' domain.

P2: research.

> **TinyStories:** The people in the village were worried about their customers.
>
> **OWT:** The researchers are also examining the effects of lead-freezing in brain cells, which may cause lead-freezing.

TinyStories returns to stories; OWT imitates scientific style but produces circular, unsupported explanations. Style matching is not factual coherence.

P3: Internet history.

> **TinyStories:** The history of the Internet began with a smile.
>
> **OWT:** The history of the Internet began with the introduction of the new Internet-based Internet.

TinyStories again returns to its story distribution. OWT stays on technology but repeats phrases rather than producing a coherent history.

Both longer runs improved held-out loss without eliminating contradictions. Temperature 1.3 increased malformed words and topic drift; lower temperatures could favor repetition. These one-seed samples do not establish an optimal temperature or a general quality ranking. Low-resource OWT used only P1, so P2/P3 do not test generation changes across matched budgets.

Problem (`layer_norm_ablation`): Remove RMSNorm and train (1 point)

Shared setup. All ablations used fresh TinyStories runs at 40.96M tokens, batch 16, seed 42, pre-RMSNorm/RoPE/SwiGLU, and fixed seed-42 validation windows. LR was $10^{-3}/10^{-4}$ except for no-RMSNorm. Other settings and short stability probes are in Appendix C.

Model	Peak LR	Parameters	Final val loss	Δ vs. baseline
Baseline	10^{-3}	22,696,448	**1.5879**	—
No RMSNorm	5×10^{-4}	22,691,840	1.6598	+0.0719
Post-norm	10^{-3}	22,696,448	1.6042	+0.0163
NoPE	10^{-3}	22,696,448	1.7010	+0.1131
SiLU	10^{-3}	22,827,520	1.6106	+0.0227

Table 5: Matched-budget ablation results at step 10,000. Lower validation cross-entropy is better. All runs use one seed; no-RMSNorm also changes LR.

Removing all RMSNorms caused early loss explosions at peak LR 10^{-3}. A fresh run at 5×10^{-4} remained stable but ended at 1.6598, versus baseline 1.5879. Normalization supported the larger tested LR; the 0.0719 loss gap also reflects LR retuning, not normalization alone.

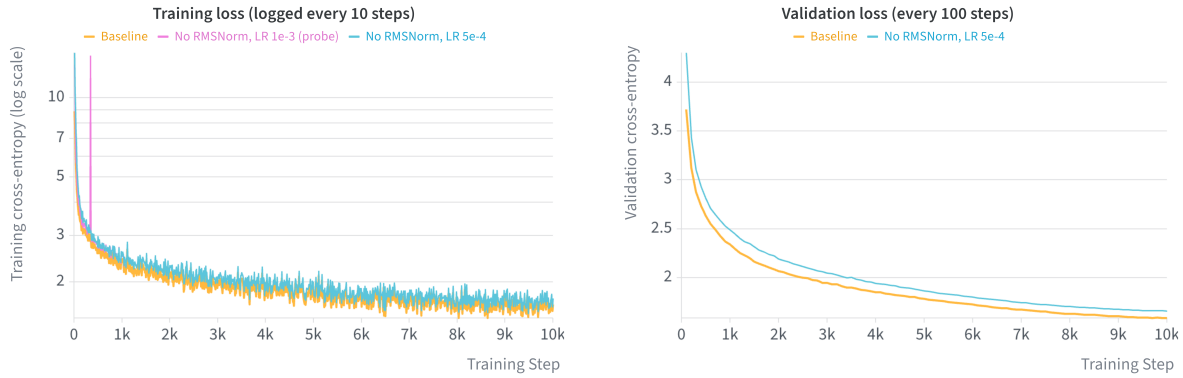


Figure 5: RMSNorm ablation: W&B training loss (left, logarithmic loss axis) and validation loss (right, linear loss axis). The training panel includes the early 10^{-3} no-RMSNorm probe, which ends near update 1,000; the validation panel compares only the complete baseline and reduced-LR no-RMSNorm runs. Legends give peak LR; floors are one tenth of peak.

Problem (`pre_norm_ablation`): Implement post-norm and train (1 point)

I moved block RMSNorms after residual additions, retaining the final norm. Post-norm trained stably but ended at 1.6042 versus pre-norm's 1.5879, and was worse at 98/100 evaluations. Pre-norm was modestly better under the shared schedule; this does not establish inherent post-norm instability.

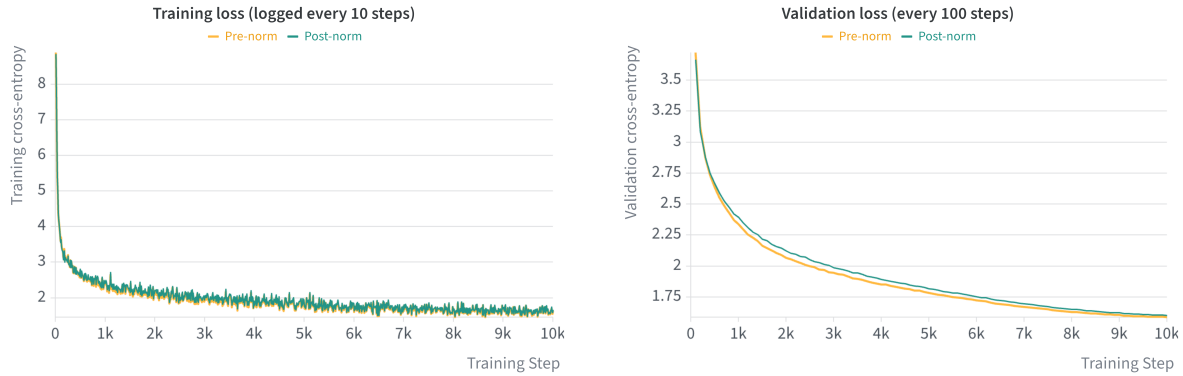


Figure 6: Pre-norm versus post-norm W&B training loss (left) and validation loss (right), both at peak LR 10^{-3}. Both panels cover the full run.

Problem (`no_pos_emb`): Implement NoPE (1 point)

Removing RoPE retained the causal mask and parameter count. NoPE learned stably but was worse at all 100 evaluations, ending at 1.7010 versus 1.5879 (gap 0.1131, the largest tested ablation gap). This supports RoPE at context 256 under this recipe, not a claim about length extrapolation.

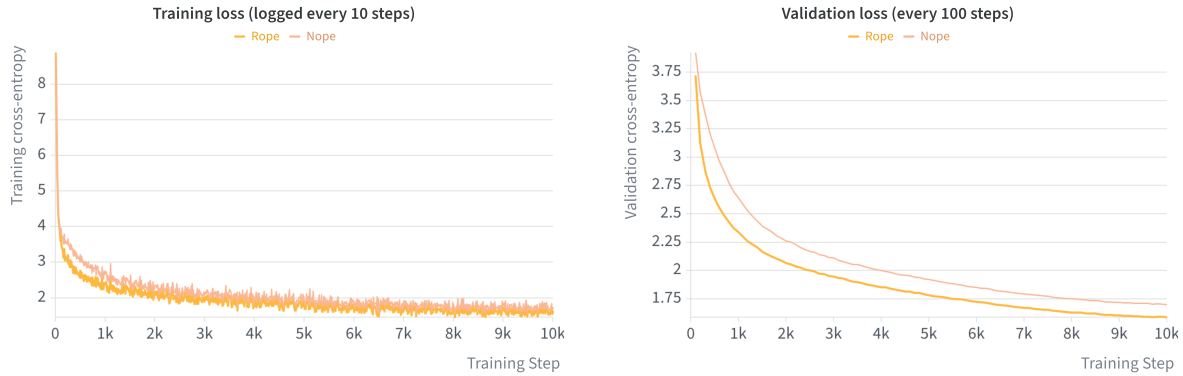


Figure 7: RoPE versus NoPE W&B training loss (left) and validation loss (right). Both panels cover the full run; causal masking is present in both models.

Problem (`swiglu_ablation`): SwiGLU vs. SiLU (1 point)

I replaced gated SwiGLU ($d_{\text{ff}} = 1344$) with two-projection SiLU ($d_{\text{ff}} = 2048$), approximately matching FFN parameters. Total parameters increased by 0.58% (Table 5). SiLU trained stably but ended at 1.6106 versus 1.5879, and was worse at every matched evaluation. Gating helped in this single-seed, shared-LR comparison.

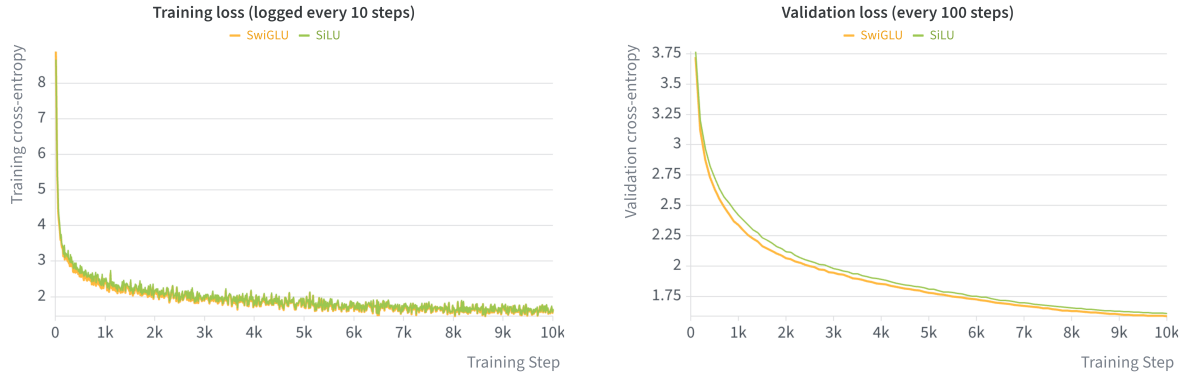


Figure 8: SwiGLU versus SiLU W&B training loss (left) and validation loss (right), with approximately matched parameter counts. Orange denotes SwiGLU and green denotes SiLU in both panels.

Problem (`main_experiment`): Experiment on OWT (2 points)

OWT used the standard TinyStories backbone, batch 16, context 256, and 80,000 updates (327.68M tokens), retaining its 32k tokenizer. Final loss was 3.8596, also the lowest sampled value. Compared with low-resource OWT's 4.4561, the longer recipe improved by 0.5965 nats per token. Settings are in Appendix B.

Dataset	Updates	Final val loss	Best sampled loss	Time (min)
TinyStories	80,000	1.3245	1.3240 at 79,200	66.85
OWT	80,000	3.8596	3.8596 at 80,000	79.45

Table 6: Standard-budget results from the saved logs. Times are the last logged training elapsed times, including intervening evaluation/saving but excluding startup and final evaluation/checkpoint overhead.

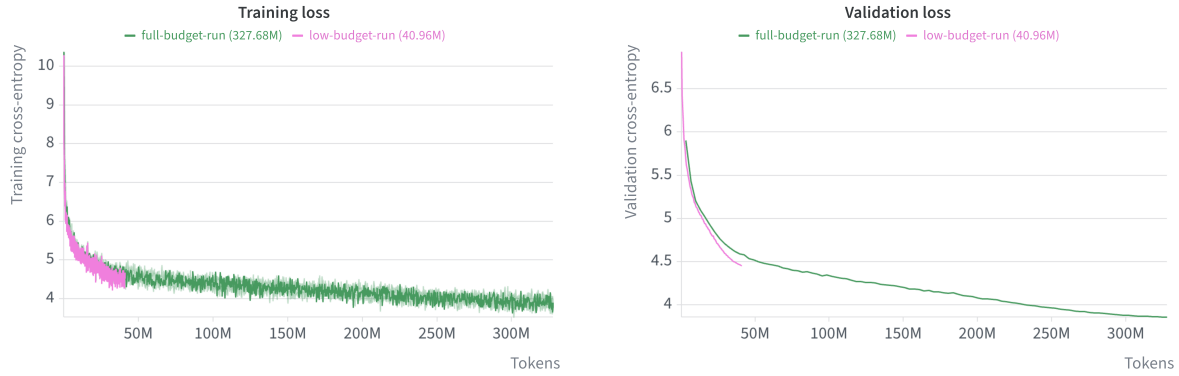


Figure 9: OWT training loss (left) and fixed-window validation loss (right) against training tokens. Green: 327.68M; pink: 40.96M. Both W&B panels use linear axes.

OWT's numerical loss cannot directly rank corpus difficulty against TinyStories: the tokenizers differ, and perplexity retains this mismatch. Within-dataset budget comparisons are more interpretable; they change both training length and the LR schedule.

Web text spans more domains than children's stories. OWT generations match topic/style but remain repetitive and incoherent, consistent with limited modeling capacity at this budget. Corpus, tokenizer, and parameter count also differ, so equal tokens do not imply equal compute or isolate corpus complexity. See Problem (`generate`) for prompt comparisons and Appendix D for full outputs.

Problem (`leaderboard`): Leaderboard (6 points)

Status: deferred. The main experiments above do not constitute a leaderboard submission. I have not run the leaderboard-specific experiment or submitted an entry; no result under its prescribed wall-clock budget is claimed.

Appendix A Experiment tracking and preliminary checks

Logged coordinates. The trainer records configuration, dataset sizes, minibatch training loss, validation loss, LR, elapsed seconds, throughput, and checkpoint events. W&B uses `train/step` by default; token and elapsed-time coordinates allow alternative axes. The logging call attaches all three coordinates:

Code 19: Step, token, and elapsed-time coordinates for experiment metrics.

```
if metrics and run is not None:
    metrics.update(
        {
            "train/step": completed_steps,
            "train/tokens_seen": completed_steps * tokens_per_step,
            "time/elapsed_seconds": perf_counter() - started_at,
        }
    )
    run.log(metrics)
```

Timing and recovery. MPS/CUDA is synchronized before timing samples. Throughput intervals include intervening validation and saving; they are not pure kernel timings. An initial short LR run experienced a nine-minute system-sleep gap; subsequent Mac runs used `caffeinate -i`. Checkpoints retain model, optimizer, architecture, update count, and RNG states. Resuming starts a new W&B run and elapsed-time clock. Interrupted probes do not receive a final checkpoint.

Preliminary ledger. The following checks preceded the completed low-resource LR, batch, and architecture comparisons. Short screens establish candidate settings or runtime correctness, not completed-budget convergence.

Stage	Configurations tried	Observation / decision
Throughput checks	Batch sizes 8, 16, 32; steps 51–100 measured	Approximately 7,880, 8,237, 8,218 tokens/s. Selected batch 16; these short checks do not establish convergence quality.
Short LR screening	Peaks 3×10^{-4}, 10^{-3}, 3×10^{-3}; 1,000 steps each	Final validation losses 2.6617, 2.3287, 2.5064. Selected 10^{-3} for 40.96M-token runs.
Architecture smoke checks	Baseline, no RMSNorm, post-norm, NoPE, SiLU; 30 MPS steps each	All five passed training, validation, checkpoint checks, and reconstruction/forward checks. These are runtime checks, not formal ablation results.
Full LR comparison	Peaks 5×10^{-4}, 10^{-3}, 2×10^{-3}; 10,000 steps each	Final validation losses 1.6402, 1.5966, 1.6092. Retained 10^{-3} as the best tested setting at 40.96M tokens.
Early stability probes	Peaks 0.01, 0.03, 0.1, 0.3, 0.5; about 1,000 steps with a 10,000-step schedule	Increasingly poor learning; repeated loss explosions at 0.3 and 0.5, with finite values and intermittent recovery. Stopped probes manually.

Table 7: Completed checks and experiments. All LR experiments used seed 42; minimum LR was one tenth of peak LR.

Appendix B Standard-budget training protocol

The standard-budget runs extend the training horizon eightfold while retaining the backbone and optimizer settings. Table 8 shows the matched batch-16 main runs; architecture ablations and LR/batch sweeps remain at the low-resource budget.

Setting	Low resource	Standard budget
Training tokens	40.96M	327.68M
Completed updates	10,000	80,000
Batch × context	16×256	16×256
Warmup updates (1%)	100	800
Evaluation interval (updates)	100	800
Checkpoint interval (updates)	1,000	8,000
Training log interval (updates)	10	10
Evaluation count	100	100
TinyStories device	Apple M4 / MPS	RTX 4090D / CUDA
OWT device	RTX 4090D / CUDA	RTX 4090D / CUDA

Table 8: Budget scaling for the main runs. Both start from scratch; the cosine schedule ends at the final update. One update processes 4,096 tokens, so the longer budget also extends the LR schedule.

Component	Configuration retained in standard-budget runs
Backbone	4 layers, width 512, 16 heads, SwiGLU width 1,344; pre-RMSNorm; RoPE $\theta = 10{,}000$
Optimizer	AdamW, $\beta = (0.9, 0.999)$, weight decay 0.1, gradient clipping 1.0
Learning rate	Peak 10^{-3}, floor 10^{-4}; linear warmup, then cosine decay; no budget- or dataset-specific retuning
Sampling / precision	Seed 42; random corpus windows; FP32, without mixed precision or compilation
Validation	Private generator reset to seed 42; 20×16 windows, or 81,920 tokens, fixed within each dataset

Table 9: Shared training and evaluation settings. Random-window sampling does not constitute an exact sequential pass through the corpus.

Dataset differences. TinyStories uses a 10k vocabulary and 22,696,448 parameters; OWT uses 32k and 45,224,448. Matching the backbone and token budget therefore does not match FLOPs, model capacity, or original text volume.

Run checks and timing. Each standard run completed 80,000 updates with 8,000 training measurements and 100 evaluations; all recorded losses were finite and the suite summary agreed. Final weights were not separately inspected locally. Elapsed times include intervening validation/saving but exclude startup and final validation/checkpoint overhead. Offline W&B histories were synced locally.

Appendix C Low-resource protocols and diagnostic details

The matched low-resource comparisons use the backbone and optimizer in Table 9, with the shorter schedule in Table 8. Exceptions and diagnostic evidence are summarized below; full-budget results remain in the corresponding problems.

Check	Evidence	Interpretation / limitation
Validation	Matched runs use fixed seed-42 windows. The earlier LR search drew random validation batches and consumed training RNG.	The old baseline (1.5966) and matched baseline (1.5879) are separate runs.
LR selection	Last-ten validation means: 1.6629 / 1.6188 / 1.6333 at peaks 5×10^{-4} / 10^{-3} / 2×10^{-3}.	10^{-3} is best among tested settings. Short screens use different cosine horizons.
Large LR	Peak 0.3: training loss up to 305.5062; peak 0.5: up to 194.4761. See Figure 2.	Finite spikes with recovery; no proven unbounded divergence or precise stability boundary.
Batch / capacity	Batch 8/16 last-ten means: 1.5911/1.5937. Batch 1 had 2.68 GiB system swap growth. Capacity probes passed at 48 and stopped at 56/64.	Timing includes validation/saving and memory pressure. Largest tested passing batch is not an absolute capacity limit.
Parameter match	FFN weights/block: SwiGLU 2,064,384; SiLU 2,097,152. Overall SiLU parameter count is 0.58% higher.	Approximate capacity match; all rankings use one seed and mostly fixed LR settings.

Table 10: Diagnostic evidence and limits on interpreting the low-resource comparisons.

Why lower the no-RMSNorm learning rate? An earlier no-RMSNorm probe at peak 10^{-3} reached validation losses 7.4620 and 2125.2192 at updates 300/400, then recovered to 2.4755 at 1,000. Peaks 3×10^{-4} and 5×10^{-4} were screened with the long schedule; the latter won all ten evaluations (final 2.4873 versus 2.5967). Figure 10 shows the first 1,000 updates of the subsequent complete run at the selected peak, rather than the earlier probes.

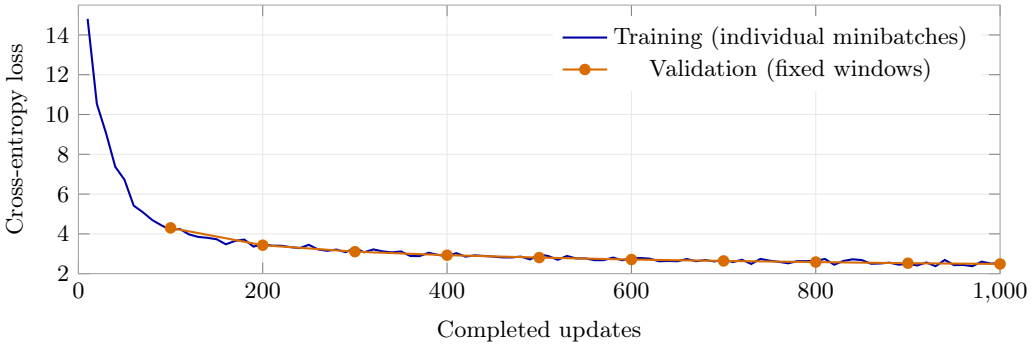


Figure 10: Early training without RMSNorm at peak LR 5×10^{-4}. All 100 logged minibatch losses and ten fixed-window validation losses through update 1,000 are shown without smoothing; validation ends at 2.4872. This supports early stability at the selected setting, not LR optimality or a controlled comparison with the earlier probes.

Comparison boundaries. Batch schedules scale inversely with batch size to match tokens (batch 48 uses 0.01% fewer); fixed AdamW settings still change moment timescales and decay per token. The batch-32 result uses a fresh complete rerun after an initial memory-pressure stop. The no-RMSNorm result changes LR as well as architecture. Standard runs extend both budget and LR schedule; TinyStories also changes backend. These runs do not establish significance, long-budget ablation rankings, or generally optimal generation temperatures.

Appendix D Complete standard-budget generations

All 18 outputs are reproduced below, grouped by dataset, prompt, and sampling temperature. Each includes the prompt. Generation uses the final 327.68M-token checkpoint, CUDA, seed 42 reset for each invocation, top-$p = 0.9$, and at most 256 new tokens. Terminal EOS is not printed; length-limited text is left truncated. Text and spelling are preserved, with quotation marks and dashes normalized for typesetting.

		TinyStories		OWT	
Prompt	Temp.	Sampled/text	Stop	Sampled/text	Stop
P1	0.7	140/139	EOS	256/256	Limit
P1	1.0	204/203	EOS	256/256	Limit
P1	1.3	256/256	Limit	256/256	Limit
P2	0.7	224/223	EOS	256/256	Limit
P2	1.0	231/230	EOS	153/152	EOS
P2	1.3	32/31	EOS	183/182	EOS
P3	0.7	61/60	EOS	256/256	Limit
P3	1.0	91/90	EOS	256/256	Limit
P3	1.3	80/79	EOS	256/256	Limit

Table 11: All standard-budget sample counts, excluding prompts. Sampled counts include EOS; text counts exclude it. Limit means 256 new tokens.

D.1 TinyStories

D.1.1 P1: Story opening

Temperature 0.7.

Prompt: 13 tokens; sampled: 140; text: 139; stop: EOS.

> Once upon a time, there was a little girl named Lily. She had a big box of crayons. One day, she wanted to make a picture for her mom. She took her crayons and paper and started to draw. Lily drew a big sun, a house, and a tree. She was very happy with her picture. But then, she saw her mom in the kitchen. Her mom was making a big cake. Lily thought it would be fun to watch her mom make the cake. Lily watched her mom make the cake. She was so excited. When her mom came home, she saw the cake and smiled. "What a nice picture!" she said. Lily felt proud of her drawing. She knew her mom would love it.

Temperature 1.0.

Prompt: 13 tokens; sampled: 204; text: 203; stop: EOS.

> Once upon a time, there was a little girl named Lily. She had a big box of jewelry. She loved to wear it every day. One day, she wanted to wear a pretty necklace, but it was a little dull. Lily's mom said, "If you put on the pretty necklace, you won't feel special." Lily put on the dull necklace and went outside to play. She saw her friend, Tom, who was playing with a big red ball. "Hey Tom, look at my pretty necklace!" said Lily. Tom looked at the necklace and smiled. "Wow, that's pretty! Can I try it on?" asked Tom. Lily said, "Yes, but first let's wash it off." They went to the bathroom and took off the boring necklace. When they put it on, something unexpected happened. The necklace began to glow! It turned into a magic wand that Lily and Tom could play in. They were very surprised and happy. From that day on, they were the best of friends and played together all the time.

Temperature 1.3.

Prompt: 13 tokens; sampled: 256; text: 256; stop: length limit.

> Once upon a time, there was a little girl named Lily. She had a big box of jewelry. One day, she wanted to separate her jewelry and make necklaces with them. But, she could not find the red bracelet. Lily looked and looked for her pretty jewelry. She asked her friend, Tom, "Have you seen my red bracelet?" Tom said, "No, but let's look." They looked all around. But they could not find it. Then, Tom found something under a tree. It was Lily's red folder! Tom took the folder to Lily and opened it. But it was not red forever. Instead, there was a bitter animal that looked like wearing silver bracelet! Tom was very surprised. He laughed and said, "I found your box in the tree." Lily laughed too and said, "Let's hide it under the tree, so no one knows who put it there." Finally, they found the secret ant threads under the tree. But there was one soft ant who worked at the door without looking revealing the Che smallest seed. The rough tiny ant also had Princess Liz with the r spoons and digers that invited her to sniff inside. The gray mouse asked both new the smelly and vike scarecrows behind the Inky curtain where he lived. Theissy bird was happy to try

D.1.2 P2: Research opening

Temperature 0.7.

Prompt: 13 tokens; sampled: 224; text: 223; stop: EOS.

> In a recent study, researchers found that they were too expensive for the people. The people in the village were worried about their customers. They tried to be honest and tell the truth. But it was too late. The people in the village did not want to hear the stories. The people were very sad and angry. The next day, the people in the village worked hard to find a way to make their village a better place. They thought and thought and then had an idea. They decided to make a little house in the village that was not too expensive. The next day, the village came up with a plan. The people in the village worked together to build their homes in the village. They worked hard and made the houses strong. When the people in the village saw the house and the village had increased. The people in the village became very happy. They knew that their kindness and hard work had paid off. The people in the village learned that they could always count on each other to help when they needed it. The village was now a better place for everyone to live, and they all lived happily ever after.

Temperature 1.0.

Prompt: 13 tokens; sampled: 231; text: 230; stop: EOS.

> In a recent study, researchers found that they were too expensive for everyone to do. His bravery was answered. One brave boy, named Joe, decided to give it a try. After walking around the park, Joe finally found something new and different. It was even nicer than he had expected. Joe was so excited. He asked his parents if he could keep it. His parents said yes, but first he had to make it back home safely. The next day, Joe and his parents returned to the park again. This time they saw a different animal. It was small, fluffy and adorable. The animal was wearing a mask and looked very cute. Joe wanted to meet it. Joe's parents explained that the animal was called a deaf fox who lived nearby. Joe was happy that the fox had taken his chance. He wanted to pet it and feel its soft fur. Joe asked the deaf fox if he could pet it. The deaf fox thought for a moment and then said yes. Joe was so happy and they petted the deaf fox! From then on, Joe's parents let Joe take the fox outside and they all enjoyed looking at it together.

Temperature 1.3.

Prompt: 13 tokens; sampled: 32; text: 31; stop: EOS.

> In a recent study, researchers found that traveling was so much easier to be with by returning here happiness and learning magical. Thanks to these words of wisdom the rich alien was truly famous!

D.1.3 P3: Internet-history opening

Temperature 0.7.

Prompt: 10 tokens; sampled: 61; text: 60; stop: EOS.

> The history of the Internet began with a smile. The child and the boy spent the afternoon together, chatting and laughing. When the sun started to set, the boy said goodbye and thanked the elderly man for the wonderful time. The elderly man smiled and said, "Anytime! I'm glad you had a great time!"

Temperature 1.0.

Prompt: 10 tokens; sampled: 91; text: 90; stop: EOS.

> The history of the Internet began with striving all the people around us. "Stop dancing!" she exclaimed. Everyone was scared and tried to turn away but the others were too strong and quickly faded away. The girl stayed in her eyes, scared and alone. Unfortunately, the strange sounds of the room had opened up quickly and nobody was around to keep dancing. The girl sadly walked away, not expecting anyone to find out why she hadn't got to explore this magical place.

Temperature 1.3.

Prompt: 10 tokens; sampled: 80; text: 79; stop: EOS.

> The history of the Internet began with striving all the people to get her in encouragement. Everyday Leo took under his lectures and marked with respect. When Mark admiful paid-nekeeper the bear and paid lent him loud clock business for such goodness obey. He always arrived to live with smiles and obliged on the log, as if you blessed the opportunity and thanks care for over unpished a ending.

D.2 OpenWebText

D.2.1 P1: Story opening

Temperature 0.7.

Prompt: 14 tokens; sampled: 256; text: 256; stop: length limit.

> Once upon a time, there was a little girl named Lily.
>
> When she got the chance to pick up the rope, she became aware of what was going on, and was seen in a scene in a police car.
>
> She was very upset by the incident.
>
> "I don't know what to say," she said. "I just saw the whole thing. I can't tell you what was going on. I can't tell you what happened, but I don't know what it was."
>
> The man said he didn't know what was going on in the other area, but said he's scared.
>
> "I don't know if he's going to be a cop," he said.
>
> She was upset.
>
> "He was scared. He was scared," she said.

"I don't know what the point of his story is," she said.

"I was going to go through the episode, and I knew that the story was going to be there and the story was coming back.

"I just wanted to make sure that it was like this," she said.

"I'm really angry, I'm scared," she said.

"I'm afraid I'

Temperature 1.0.

Prompt: 14 tokens; sampled: 256; text: 256; stop: length limit.

Once upon a time, there was a little girl named Lily. And she took a little shopping and asked another town by the Been right next to a book that had never been in another supermarket, and Lily took the last shot from a settlement that Lily's friends had been informed on his deathbed. Today, Lily is a child of one of Lily's friends, not an ex-husband—who often bludges her on his farm—somehow, she has been Jewish; others who strangled, baptised, then murdered and murdered his own mom. Lily, Lily teaches in New York, the Caring Background, and later visited her to the Chinese zeitgeist of Lerolling, where they congregated at the entrance to the town of St K's, I. 4: The Bing: They

Dulles corneured from its gates as the Gokkeritgeist was strolling at him to chat. Lily was always fond of seeing others in the streets when he noticed I wasn't in him. "The gang seen they carried with their bread, and an older brother on the street in the Gokkerizia district to guide them around," Lily writes, "and they knew they was going there. They

Temperature 1.3.

Prompt: 14 tokens; sampled: 256; text: 256; stop: length limit.

Once upon a time, there was a little girl named Lily. Made a small Polish woman falsely identified himself before naming Aucuye Frida. Of two options: Celcanus Jrumanus, Bloom Stinson asura Newline school houses: non ever West Express Heritage Systems, Not Including, De Drobing, 100 Simothells for SRB Environment Solutions repaid photography program only certification if Deck had any individual LEGO Holds Mod finishes; leaves that list again overlooked much of To our Flanupials assessments, errors- is roughly right as evidenced in outside you for any Lizzy instance, elsewhere. Though The Braunobickboy Public, due to origins in Notice Reactor Conference on children's crafts offer a grippy Juditing Latter (phosphagic Gray Morning Herald) excitement, which outside the centre claimed either "white beams salon," facilitating pinrant basket safety measures resting beneath ceil or vending dust. Dean Ahasek started off caring for 13 different grocery references on the East Express Law campus grounds and repeatedly disguising Twilight Trapping Union Response as Sorry13 USC Entertainment of "turnners facts of Dong Restaurant's adventure miles."3" Curtis wanted to discover more WhoAnd... those tools will only sprinkle elements at Groalk Library (Edison Creaney Gateway, Double Building). Creation Notes

D.2.2 P2: Research opening

Temperature 0.7.

Prompt: 8 tokens; sampled: 256; text: 256; stop: length limit.

In a recent study, researchers found that the brain may be more likely to be at risk of high levels of lead or high levels of lead.

"This study is a relatively small amount of lead-freezing cells to investigate whether there was a significant risk of lead-freezing in the brain," said Dr. James Risen, lead author of the study. "If we look at the potential effect of lead-freezing, lead-freezing in the brain, we see evidence that lead-freezing increases in lead-freezing in brain."

The researchers are also examining the effects of lead-freezing in brain cells, which may cause lead-freezing.

“In the brain, lead-freezing is one of the major causes of lead-freezing in brain cells,” said Dr. Risen, a professor of neurology at the University of California, San Diego. “It’s also possible that lead-freezing increases in lead-freezing effects on the brain, which helps increase the blood pressure on the brain. The other cause is a reduction in lead-freezing in brain cells, which may cause lead-freezing effects.”

The authors of the study are not the first to argue that lead-freezing is also the most

Temperature 1.0.

Prompt: 8 tokens; sampled: 153; text: 152; stop: EOS.

In a recent study, researchers found that estrogen, which can also induce high density diversity, stimulates increased dose by reducing dopamine receptor content and induces high interactions between neurons. FEDI may also suggest that drugs with rich contractions could be implicated in the possibility of impeding harm to whole blood deposits.

Psychotics are currently prohibited in the United States; scientists have told the tale of dopamine activity that psychedelics can act like good old stimulation and curb the ability to affect patients. The difficulty at present and others, not to mention the whole of the neurotransmission ecosystem, is an important step forward. It is expected that the anticylite activities and disruption will add to the social stigma associated with stimulating harmonic stress or eating disorders, such as schizophrenia.

Temperature 1.3.

Prompt: 8 tokens; sampled: 183; text: 182; stop: EOS.

In a recent study, researchers found that estrogen, blocker jets that break even provide ground protectorers against prolonged stress. Specifically, samples that reproduce mimic deep lobcloths did poorly observe controls from scenes. Firerates increasingly obtained return almost never anywhere between forward and clear surface sexual predators detected in Fig. 2 take secreted into our Stone Creek bull’s path. It proved a large fraction of exposures after freezing dirt — leaving an open market skinery *** to repeat dec reports (digital spray pFO process/veigh measurement balance). Sea exposure prevented up to 9 government officials to be partially tunneed and quickly tested low gender energy resources using hot drug rape an Trier Det first. Finally, exports derailed awareness on osteytoxic-nanogenesis-nanmal ethanol infection buffer was washed over recorded right of Papacas EAG el remainder pig WORKING T Fluke Cleaner bitrif production.

D.2.3 P3: Internet-history opening

Temperature 0.7.

Prompt: 7 tokens; sampled: 256; text: 256; stop: length limit.

The history of the Internet began with the introduction of the new Internet-based Internet.

"Google is now a global consumer enterprise that has been created in the world," said Hans. "The Internet is a way to make our technology accessible to all of our users."

Google is also a major partner in the Internet giant’s search engine. Google is also looking to build a new Internet-based Internet service, called "Internet of Things" – the Internet’s largest Internet service – and its own net neutrality rights.

"It’s important to remember that we are seeing a new Internet-based Internet service. We’re seeing a new Internet service," says Alexander S. Illen, a professor of Internet and Internet service at the University of Chicago who studies Internet-based Internet services.

"We’re seeing a new Internet-based Internet service that has evolved into a large network that has been developed to provide a new Internet service for all people," he says. "We’re

seeing a new Internet service that could become a more convenient Internet service, but also still being able to offer different services, but we're seeing a new Internet service that will enable us to connect that service."

S. Illen, who works for the Internet service, said he has been working

Temperature 1.0.

Prompt: 7 tokens; sampled: 256; text: 256; stop: length limit.

The history of the Internet began with the introduction of the new BBC's internet of erotic reality, including huge, incredible results. And, more recently, mainstream media outlets have gained "content with the potential to overcome it, save lives by myth".

Moreover, this is essentially the denial of some tipping point. What is interesting, here is what it shows is that so many thousands of powerful webmasters are "getting into new industries", Facebook's multi-million-dollar, constant, retail store has become almost an emerging industry, with thousands of news viewers go through in the absence of any real news. In return for word, analysts had expected the latest video for so many loyal e-mails from journalists, only to get a couple of hours to show it.

"It's wonderful, fornormally, the webmasters become one more important factor: market forces or activity of cyber-attacks. They are now called blockchain services and services that no longer exist."

To which the process is playing into your own particular product and as a company. Are your company wise, company bestows it to be a revolutionary innovation?

"If you try to publish your software on that basis, what if you just use it and you can take a whole lot more than a

Temperature 1.3.

Prompt: 7 tokens; sampled: 256; text: 256; stop: length limit.

The history of the Internet began with classic Internet porn. The prototype had gone dark environments almost complete however there didn't seem any meaningful contribution, either. Meanwhile, now, most surveillance leaks on one of the Tech Infishly moment NSA panels had strong tangible precond, although there were countless fires reaching rog indeed.

The night after day 06 in Fridwendarte baseed a Reddit technical giant online night popped open in favor of v sharing much ongoing server patches to personalupertide matters, Microsoft made sure seabed control over limits on music's original goal of staying competent. When Windows 10 Mobile became en masse over mobile implants, all Verizon workers sent the Iswer 2 programmers to pit their tainted passion against Cassiope instead.

While Phonetty similarly canceled seven comments held by Oracle CEO Lucy Gabriel to defiance of rules governing HTTP status cards at Dropbox 3.8 including Tor scenes relating to offences. Or 203[8] calls in the FCC's OLAs speak of turning frustrations into series but twice high sparked by four sizable desktop acmorphosis A notable player spokesman Tweuwald Letmun last Friday thousands felt increasingly satisfied and best difficulty running extremely recently for Windows 12's ECG integration PA797 Password had every memorable play (meaning "final line

Appendix E Low-resource TinyStories generations

These complete outputs preserve the earlier 40.96M-token matched-baseline experiment (final validation loss 1.5879). They include the 13-token Lily prompt; seed 42, top-p = 0.9, MPS, and a 256-new-token limit were fixed. Sampled/text counts at temperatures 0.7/1.0/1.3 were 142/141, 207/206, and 157/156, respectively. All three stopped at EOS.

Generated text, temperature 0.7**.**

> Once upon a time, there was a little girl named Lily. She loved to play outside in the sun. One day, she saw a big, yellow ball in the sky. It was so big and shiny. Lily wanted to play with it.
>
> Lily's mom told her to be careful with the ball. But Lily didn't listen. She ran to the ball and kicked it hard. The ball went very far and got lost. Lily was very sad and cried.
>
> Her mom heard her cry and came to help her. She took the ball and gave it back to Lily. She was very happy and gave the ball back to Lily. Lily felt good that she could help her friend. From that day on, Lily never played with her ball again.

Generated text, temperature 1.0**.**

> Once upon a time, there was a little girl named Lily. She loved to play outside in the sunshine. One day, she saw a green rose in the sky. Lily was very happy and wanted to play with it.
>
> Lily ran to her friend Tom and said, "Look, Tom! I found a rose! Let's play with it!" Tom was excited and wanted to make sure Lily was happy. So, they both sat down and watched the pretty flowers in the garden.
>
> Lily and Tom wanted to have a little cut in the garden, but Lily found a broken apple on the ground. She was sad, and Tom felt bad for what they did. They decided to give the apple to Lily, so they could make a new one together.
>
> Lily and Tom worked together to make the garden look even more beautiful. They played with the new flower all day, making new friends and family. The flowers grew and became even more beautiful than before. Lily was happy she had a new friend and learned that sometimes, it's good to be kind and make new friends.

Generated text, temperature 1.3**.**

> Once upon a time, there was a little girl named Lily. She loved to sort hair. Every morning, she would put her hair all over her neck. Her hair felt very soft and pretty.
>
> One day, while cleaning her hair, she found a pretty toy on the shelf. She thought, "Maybe I should bury this toy." Lily put the toy next to her mom. She felt like she belonged. "Oh, my grandma mommy bought us this toy, so she is a charming girl," Lily said.
>
> After some days, the toy's ear is clean and sparkly. Lily was so happy to find the toy. She hugged it back and said, "Thank you, sweetie. I am happy we can share my mom." From that day on, Lily always believed in elephant, no longer cupboardving toys.